SafeNode

PRODUCT AND SECURITY ARCHITECTURE

Passkey-first identity, recovery and team secrets

Zero-knowledge design | WebAuthn PRF | Release readiness edition

What this document covers

Executive overview

SafeNode is a passkey-first identity, recovery and team secret platform. It combines phishing-resistant authentication with a zero-knowledge vault boundary: authentication proves who the user is, while client-side cryptography determines whether that user can decrypt protected data.

> *The product is designed around a practical reality: passkeys improve sign-in, but organizations still need recovery, device governance, shared operational secrets and continuity when people or hardware change.*

Core proposition	One security workspace for passkey sign-in, cryptographic vault unlock, recovery readiness, trusted devices and team secret governance.

Current implementation at a glance

Capability	Status	Evidence / boundary
Passkey authentication	Implemented	WebAuthn registration and sign-in with authenticated, user-scoped backend routes.
Passkey vault unlock	Implemented	WebAuthn PRF output derives a local KEK that unwraps the random vault key.
Recovery fallback	Implemented	Recovery kit and vault passphrase wrap the same random vault key for continuity.
Local secret persistence	Removed	Master password and raw vault key are memory-only; encrypted vault cache contains no raw secret.
Team workspaces	Implemented baseline	Client-encrypted team vault payloads, role-based authorization and audit events.
Android release	Release candidate	Stable signer, Credential Manager bridge and Digital Asset Links are built; production hosting is blocked.

1. Product thesis

Passkeys are becoming the default mechanism for proving identity. That transition removes a major class of phishing and credential-reuse risk, but it does not eliminate the need to protect recovery material, infrastructure credentials, emergency access records or shared team secrets.

SafeNode addresses the layer after sign-in:

- Authenticate people with passkeys and trusted devices.
- Unlock encrypted personal data without sending decryption material to the server.
- Recover access without introducing server-side key escrow.
- Govern team secrets through membership, roles and auditable change.
- Make security posture visible instead of hiding it behind settings pages.

Target users

Capability	Status	Evidence / boundary
Security-conscious individuals	Personal	Passkey-first sign-in, recovery readiness, encrypted identity records and device control.
Families and trusted circles	Continuity	Recovery material, successor planning and controlled shared access.
SMBs and remote teams	Teams	Shared operational credentials, role-based access and auditable vault activity.
Agencies and operators	High-change	Onboarding, offboarding, break-glass records and controlled customer environments.

Positioning SafeNode is not another password database. It is the continuity and secret-governance layer for a passkey-first world.

2. Security model and threat boundary

SafeNode assumes that networks, storage providers, browser extensions and even application infrastructure can be targeted. The design therefore separates authentication data from vault decryption material and minimizes the secrets available to backend operators.

Assets protected

- Personal vault entries and secure records.
- Recovery kits and continuity material.
- Team vault ciphertext and operational secrets.
- Passkey credential metadata and PRF-wrapped vault keys.
- Device sessions, membership and audit records.

Trust boundaries

Capability	Status	Evidence / boundary
Client runtime	Sensitive	Performs key derivation, wrapping, unwrapping and decryption. Raw vault key is held only for the active session.
Authenticator	Sensitive	Produces WebAuthn assertions and optional PRF output. PRF secret is never sent to SafeNode.
Backend API	Untrusted for plaintext	Stores encrypted vaults, salts, wrapped keys, public credentials and authorization state.
Database / hosting	Ciphertext only	Compromise should not reveal vault plaintext without user-controlled key material.
Recovery holder	User-controlled	Possession of the recovery kit is intentionally sufficient to restore the wrapped vault key.

Primary threats and controls

Capability	Status	Evidence / boundary
Credential phishing	Passkeys	Origin-bound WebAuthn authentication replaces reusable login secrets.
Database compromise	Zero knowledge	The server stores ciphertext and wrapped keys, not raw vault keys or PRF output.
XSS / malicious extension	Reduced persistence	No cached master password or raw vault key remains in localStorage, sessionStorage or IndexedDB.
Lost device	Recovery + revocation	Recovery kit and secondary access paths preserve continuity; device sessions can be revoked.
Authorization confusion	Fail closed	Forbidden vault access must never be interpreted as a missing vault or trigger reinitialization.

3. Cryptographic architecture

Zero-knowledge invariant	The backend never receives the vault key, master password, recovery kit or WebAuthn PRF output. It receives only ciphertext, public credential data, non-secret salts and wrapped key blobs.

Personal vault encryption

- A cryptographically random 256-bit vault key encrypts vault content with AES-256-GCM.
- A vault passphrase is processed with Argon2id using a 64 MB memory cost, three iterations and a 256-bit output.
- The resulting password-derived key wraps the random vault key; it does not directly encrypt every record.
- A separately generated recovery kit derives an independent wrapper for the same vault key.
- Every passkey PRF enrollment adds another credential-specific encrypted wrap of the same vault key.

Key material storage

Capability	Status	Evidence / boundary
Raw vault key	Client memory only	Never persisted; stripped from serialized and exported vault payloads.
Vault ciphertext	Server + encrypted local cache	AES-GCM ciphertext, IV, salt and version metadata.
Passphrase wrap	Server metadata	Wrapped vault key and IV; passphrase remains user-known.
Recovery wrap	Server metadata	Recovery-wrapped key, IV and non-secret salt; kit remains with user.
PRF wrap	Per credential on server	PRF salt, wrapped vault key and IV; PRF output remains authenticator/client-local.

Cryptographic agility

The wrapped-key model decouples content encryption from authentication methods. New passkeys, recovery mechanisms or trusted-device wrappers can be added without re-encrypting every vault entry. Revoking one wrapper does not require changing the underlying content key unless compromise is suspected.

4. Passkey-PRF vault unlock

SafeNode uses the WebAuthn PRF extension to bind a passkey to vault decryption. The passkey is not merely a UI gate: its cryptographic output is required to derive the key-encryption key used to unwrap the vault key.

1	**Authenticate** Passkey assertion verifies the user and credential.
2	**Derive locally** WebAuthn PRF output enters HKDF-SHA-256 on the client.
3	**Unwrap key** AES-GCM KEK unwraps the random vault key in memory.
4	**Decrypt vault** AES-256-GCM decrypts ciphertext for the active session.

Enrollment flow

- The user first unlocks through a passphrase, recovery kit or an already-enrolled passkey.
- SafeNode generates a stable 32-byte non-secret salt for the selected credential.
- A WebAuthn assertion requests PRF evaluation for that credential and salt.
- HKDF-SHA-256 derives an AES-256-GCM key-encryption key from the PRF output.
- The client wraps the in-memory vault key and uploads only the salt, ciphertext and IV.

Unlock flow

- SafeNode fetches PRF-ready credential metadata and encrypted vault data.
- The authenticator performs an assertion with credential-specific PRF evaluation.
- The client derives the KEK, unwraps the vault key and decrypts the vault locally.
- Unsupported browsers or authenticators degrade to passphrase or recovery-kit unlock.

Capability boundary	PRF support varies by authenticator and browser. SafeNode detects support and keeps recovery paths available rather than presenting authentication as cryptographic unlock when it is not.

5. Recovery, migration and continuity

Recovery is a first-class security feature, not an administrative bypass. SafeNode restores access by unwrapping the vault key on the client; the backend does not decrypt data or hold an escrow copy of the raw key.

Recovery model

- Recovery kit: a user-held high-entropy secret wrapping the vault key independently of passkeys.
- Vault passphrase: maintained as a fallback during migration and for unsupported PRF environments.
- Secondary passkeys: additional credential-specific wrappers can reduce single-device dependency.
- Successor workflow: waiting-period and audit controls coordinate continuity without claiming backend decryptability.

Legacy migration

Capability	Status	Evidence / boundary
Legacy passphrase vault	Existing user unlocks normally	No forced lockout or destructive conversion.
Upgrade transaction	Random vault key created	Vault content moves to wrapped-key mode and a recovery kit is generated.
Passkey enrollment	Optional after unlock	PRF-capable credential receives its own vault-key wrap.
Fallback preserved	Passphrase + recovery	PRF incompatibility never strands the user.

Non-negotiable migration guardrails

- A 403 response is access denial, never evidence that a vault is absent.
- No existing vault may be overwritten by automatic first-user setup.
- Recovery readiness is confirmed before removing any legacy sign-in or unlock path.
- All destructive transitions require explicit user confirmation and auditable state change.

6. Team secrets architecture

Team workspaces extend SafeNode from personal security into shared operational continuity. The current baseline encrypts team vault payloads client-side and enforces team membership and role permissions at the API boundary.

Current baseline

Capability	Status	Evidence / boundary
Encrypted payload	Implemented	Team records are encrypted client-side with AES-GCM before server storage.
Dedicated metadata	Implemented	Vault salt, IV, ciphertext and version are first-class database fields.
Authorization	Implemented	Owner, admin, manager, member and viewer roles map to view/edit/delete controls.
Audit events	Implemented	Membership, role and permission changes create activity records.
Per-member key wrapping	Roadmap	Replace shared passphrase access with a dedicated team vault key wrapped for each authorized member.

Appropriate team content

- Shared service credentials and customer environment access.
- Infrastructure secrets, API credentials and operational notes.
- Break-glass procedures and emergency recovery records.
- Controlled onboarding and offboarding materials.
- Secure documents whose access must follow role and team membership.

Target end state

Each team vault receives a random key. That key is wrapped separately for authorized members or devices, enabling targeted revocation, rotation and stronger separation between authorization and cryptographic access. Server-side roles remain necessary, but no longer become the sole barrier to ciphertext decryption.

7. Platform architecture

Capability	Status	Evidence / boundary
Web client	React + Vite	User experience, WebCrypto operations, passkey orchestration and encrypted local cache.
Android client	Capacitor + WebView	Bundled web assets under the canonical RP origin with Android Credential Manager support.
API	Fastify + TypeScript	Authentication, authorization, vault ciphertext transport, billing and audit endpoints.
Data layer	Prisma + PostgreSQL	Users, ciphertext, devices, passkeys, teams, subscriptions and immutable activity metadata.
Identity standard	WebAuthn / FIDO2	Origin-bound public-key credentials, device transports and PRF extension metadata.
Cryptography	WebCrypto + hash-wasm	AES-256-GCM, HKDF-SHA-256 and Argon2id client-side key derivation.

Core product surfaces

- **Identity Vault:** encrypted personal records, passkey readiness and access state.
- **Recovery Center:** recovery kit, fallback readiness and successor continuity.
- **Trusted Devices:** registered devices, sessions, approval, revocation and device-limit handling.
- **Team Secrets:** shared encrypted workspaces, invitations, roles and audit.
- **Security Posture:** breach signals, weak credentials, risky devices and recovery gaps.
- **Reports and Audit:** session activity, access changes and exportable operational evidence.

API-origin discipline

Web development uses the Vite proxy. Production web and Android builds use explicit HTTPS API origins. Android release CI rejects missing, non-HTTPS or trailing-slash mobile origins, preventing a packaged app from silently targeting localhost or an unintended proxy.

8. Android trust and release chain

The Android release path treats package signing, domain association and API origin as one trust chain. A valid APK alone is insufficient: the package, signer, relying-party domain and Digital Asset Links declaration must agree.

Capability	Status	Evidence / boundary
Package identity	com.safenode.mobile	Stable Android application ID.
Release signer	Stable RSA 4096-bit certificate	Keystore remains outside Git and is restored into CI from protected secrets.
Digital Asset Links	Implemented in source	Only the production signer fingerprint is authorized for safe-node.app.
Credential Manager	Implemented	AndroidX WebKit enables WebView Web Authentication support when available.
Canonical origin	Implemented	Bundled assets are served under https://safe-node.app for WebAuthn RP alignment.
Download gate	Closed	Download remains disabled until live domain, API, APK and device checks pass.

Release gate

- Restore production hosting and verify the API health endpoint.
- Serve /.well-known/assetlinks.json over HTTPS with status 200, JSON content type and no redirect.
- Build the release APK in CI and verify its certificate fingerprint against repository configuration.
- Register a passkey, unlock by PRF, test passphrase and recovery fallback, and inspect browser storage.
- Publish the newly signed artifact, then explicitly enable the Android download flag.

Current blocker

The Vercel team is blocked for fair-use limits. safe-node.app returns 402 and api.safe-node.app is not attached to a healthy backend. The retired v0.1.3 APK uses a different signer and must remain unavailable.

9. Security controls and engineering quality

Capability	Status	Evidence / boundary
Secret persistence	Hardened	Local master-password caching removed and sensitive keychain entries blocked/purged.
Serialization hygiene	Hardened	All underscore-prefixed transient vault fields are stripped before encrypting or exporting.
Randomness	Fail closed	Cryptographic operations stop when secure random generation is unavailable.
Password derivation	Argon2id	Memory-hard derivation for passphrase wrappers; legacy verification compatibility retained where needed.
Credential ownership	Scoped	Passkey wrap updates require authentication and user + credential ownership; zero-match updates return 404.
Test database safety	Guarded	Destructive backend tests refuse non-test database names and fail fast when PostgreSQL is unreachable.
CI gating	Enabled	Frontend and backend suites run against defined build and database conditions.

Verified branch baseline

- Frontend: 11 test files, 60 tests passing; type-check and production build passing.
- Backend: 23 suites, 151 tests passing; type-check passing against a dedicated test database.
- Android: signed release APK built and verified; Credential Manager support logged on an Android 16 emulator.
- Repository: release workflow verifies signer and Digital Asset Links before publishing Android artifacts.

Independent assurance boundary

These controls and test results are engineering evidence, not a third-party certification. SafeNode should not claim SOC 2, ISO 27001, FIDO certification, penetration-test clearance or regulatory compliance until the corresponding independent process is complete.

10. Readiness, risks and roadmap

Release readiness

Capability	Status	Evidence / boundary
Application code	Ready for controlled validation	Core passkey-PRF and zero-knowledge flows are implemented and test-green.
Production hosting	Blocked	Vercel team fair-use block prevents new deployments and disables current production URLs.
Android distribution	Gated	New signer works locally; live DAL and a newly published APK are still required.
Operational security	Action required	Previously exposed JWT and encryption secrets require confirmed rotation and history purge.
Team cryptography	Baseline only	Current encrypted passphrase vaults should evolve to per-member wrapped random keys.
External assurance	Not started	Independent penetration testing and formal control evidence remain pre-production milestones.

Prioritized roadmap

- **P0 - Restore and secure production:** resolve hosting block, rotate leaked secrets, deploy API and web, publish live DAL.
- **P0 - Complete Android gate:** issue newly signed APK and run real-device passkey, PRF and recovery checks.
- **P1 - Multi-device PRF:** validate authenticators, synced passkeys and cross-browser fallback behavior.
- **P1 - Team wrapped keys:** add dedicated team keys, per-member wraps, targeted revocation and rotation.
- **P1 - Security assurance:** external review, threat-model workshop, dependency/SBOM discipline and incident exercises.
- **P2 - Enterprise controls:** policy enforcement, lifecycle automation, richer audit exports and directory integration.

11. Buyer and reviewer due diligence

SafeNode should be evaluated on concrete evidence rather than security adjectives. The following questions define the expected diligence package.

Cryptography

Where are keys generated? Which secrets leave the client? How are wrappers revoked? What happens when PRF is unsupported?

Identity

Which WebAuthn verification rules are enforced? How are challenges expired? How are sign counters and credential ownership handled?

Recovery

Who can trigger recovery? What user-held material is required? Can support staff decrypt or bypass waiting periods?

Teams

Does API authorization match cryptographic access? How are members removed? Which operations are audited?

Operations

How are production secrets rotated? What is the rollback plan? Which alerts detect auth and vault failures?

Assurance

Which tests gate releases? Has an independent review occurred? Which claims are certified versus self-attested?

Evidence-first posture	A professional security product should publish architecture decisions, limitations, release gates and remediation status with the same clarity as features.

References and source basis

Source	Use
SafeNode source	Current security/p0-hardening branch, commits through 853a2bd, reviewed 15 July 2026.
SafeNode architecture	docs/PASSKEY_FIRST_ARCHITECTURE.md and implemented PRF, recovery, vault and team code paths.
Android release	docs/android-release.md and GitHub release workflow.
FIDO Alliance, State of Passkeys 2026	https://fidoalliance.org/the-state-of-passkeys-2026-global-consumer-and-workforce-report/
FIDO Alliance, Passkey Index 2025	https://fidoalliance.org/passkey-index-2025/
Verizon, 2025 DBIR credential research	https://www.verizon.com/business/resources/articles/credential-stuffing-attacks-2025-dbir-research/
IBM, Cost of a Data Breach 2025	https://www.ibm.com/reports/data-breach
Android, Credential Manager in WebView	https://developer.android.com/identity/sign-in/credential-manager-webview
Android, Digital Asset Links prerequisites	https://developer.android.com/identity/credential-manager/prerequisites

This document is a product and engineering brief. It is not legal advice, a compliance attestation or an independent security certification.